



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

LABORATORIO DE MICROCOMPUTADORAS

**Práctica 02 Programación en ensamblador,
direccionamiento indirecto**

Grupo de laboratorio: 02

Profesor: Ing. Moisés Meléndez Reyes

Brigada: 04

Integrantes:

- Hernandez Acosta Mauricio Gabriel 321121051
- Pérez Olvera Alexis Abraham 321150503
- Sierra Garcia Mariana 320301238



Fecha de entrega: Domingo 20 de septiembre del 2026

Semestre: 2027-1

Índice

1. Objetivo	2
2. Actividades	2
2.1. Ejercicio 1	2
2.1.1. Pseudo codigo	2
2.1.2. Desarrollo	2
2.1.3. Analisis de resultados	3
2.2. Ejercicio 2	4
2.2.1. Pseudo codigo	4
2.2.2. Desarrollo	4
2.2.3. Analisis de Resultados	5
2.3. Ejercicio 3	6
2.3.1. Pseudo codigo	6
2.3.2. Desarrollo	6
2.3.3. Analisis de Resultados	8
2.4. Ejercicio 4	9
2.4.1. Pseudo codigo	9
2.4.2. Desarrollo	9
2.4.3. Analisis de Resultados	10
2.5. Ejercicio 5	11
2.5.1. Pseudo codigo	11
2.5.2. Desarrollo	11
2.5.3. Analisis de Resultados	13
2.6. Ejercicio 6	14
2.6.1. Pseudo codigo	14
2.6.2. Desarrollo	14
2.6.3. Analisis de Resultados	16
2.7. Ejercicio 7	17
2.7.1. Pseudo codigo	17
2.7.2. Desarrollo	17
2.7.3. Analisis de Resultados	19
3. Conclusiones	20
3.1. Hernandez Acosta Mauricio Gabriel	20
3.2. Perez Olvera Alexis Abraham	20
3.3. Sierra Garcia Mariana	20

1. Objetivo

Programar las variantes del modo de direccionamiento indirecto existentes para los procesadores ARM.

2. Actividades

2.1. Ejercicio 1

Escribir, comentar, compilar y comprobar el funcionamiento del siguiente programa.

2.1.1. Pseudo codigo

Algoritmo 1: Llenado de arreglo con valores consecutivos

Definir ARREGLO[16];

CONTADOR $\leftarrow$ 0;

Mientras CONTADOR < 16 hacer;

 ARREGLO[CONTADOR] $\leftarrow$ CONTADOR;

 CONTADOR $\leftarrow$ CONTADOR + 1;

Finalizar programa;

2.1.2. Desarrollo

```
1  .data
2  i:  .skip 64           @ reservar 64 bytes para arreglo i
3
4  .text
5  .global main
6
7  main:
8  ldr r1, =i             @ r1 <-- direccion base del arreglo i
9  mov r2, #0             @ r2 <-- 0 (contador)
10
11 loop:
12 cmp r2, #16            @ comparar contador con 16
13 beq fin                @ si r2 == 16 terminar
14
15 add r3, r1, r2, LSL #2  @ r3 = direccion i + (r2 * 4)
16 str r2, [r3]           @ guardar r2 en i[r2]
17
18 add r2, r2, #1          @ r2 = r2 + 1
19 b loop                 @ repetir ciclo
20
21 fin:
22 b fin                  @ ciclo infinito
```

```

(gdb) b 15
Breakpoint 1 at 0x10090: file p2e1.s, line 15.
(gdb) r
Starting program: /home/pi/Downloads/p2e1

Breakpoint 1, loop () at p2e1.s:15
15      b loop
(gdb) i r
r0      0x0          0
r1      0x1109c     69788
r2      0x1         1
r3      0x1109c     69788
r4      0x0         0
r5      0x0         0
r6      0x0         0
r7      0x0         0
r8      0x0         0
r9      0x0         0
r10     0x0         0
r11     0x0         0
r12     0x0         0
sp      0xffffef0c0 0xffffef0c0
lr      0x0         0
pc      0x10090     0x10090 <loop+20>
cpsr    0x80000010 -2147483632
fpscr   0x0         0
tpidruro <unavailable>
(gdb) █

```

Figura 1: Debug Ejercicio 1 con dbg

r0	00000000	r0	00000000	r0	00000000	r0	00000000
r1	00000030	r1	00000030	r1	00000034	r1	00000034
r2	00000000	r2	00000000	r2	00000000	r2	00000000
r3	00000000	r3	00000000	r3	00000000	r3	00000000
r4	00000000	r4	00000020	r4	00000020	r4	00000020
r5	00000000	r5	00000000	r5	00000000	r5	00000000
r6	00000000	r6	00000000	r6	00000000	r6	00000000
r7	00000000	r7	00000000	r7	00000000	r7	00000000
r8	00000000	r8	00000000	r8	00000000	r8	00000000
r9	00000000	r9	00000000	r9	00000000	r9	00000000
r10	00000000	r10	00000000	r10	00000000	r10	00000000
r11	00000000	r11	00000000	r11	00000000	r11	00000000
r12	00000000	r12	00000000	r12	00000000	r12	00000000
sp	00000000	sp	00000000	sp	00000000	sp	00000000
lr	00000000	lr	00000000	lr	00000000	lr	00000000
pc	00000004	pc	0000000c	pc	00000018	pc	0000001c
cpsr	000001d3 NZCVI SVC	cpsr	000001d3 NZCVI SVC	cpsr	800001d3 NZCVI SVC	cpsr	800001d3 NZCVI SVC
spsr	00000000 NZCVI ?	spsr	00000000 NZCVI ?	spsr	00000000 NZCVI ?	spsr	00000000 NZCVI ?

Figura 2: CPULator Ejercicio 1

2.1.3. Analisis de resultados

El programa reservo un bloque de memoria de 64 bytes, utilizando un ciclo iterativo para almacenar 16 enteros del 0 al 15. Se empleo direccionamiento indexado con desplazamiento a la izquierda (LSL #2) para garantizar la correcta alineacion de los datos de 32 bits en memoria, finalizando la ejecucion de forma controlada mediante una condicion de parada..

2.2. Ejercicio 2

Modificar el programa de la actividad 1, para usar el direccionamiento indexado de su preferencia con el doble de datos.

2.2.1. Pseudo codigo

Algoritmo 2: Llenado de arreglo usando direccionamiento indexado

Definir ARREGLO[32];

CONTADOR $\leftarrow$ 0;

PUNTERO $\leftarrow$ direccion base de ARREGLO;

Mientras CONTADOR < 32 hacer;

 Guardar CONTADOR en la direccion apuntada por PUNTERO;

 PUNTERO $\leftarrow$ PUNTERO + 4;

 CONTADOR $\leftarrow$ CONTADOR + 1;

Finalizar programa;

2.2.2. Desarrollo

```

1      .data
2      i:  .skip 128           @ 32 enteros (32 x 4 bytes)
3
4      .text
5      .global _start
6      _start:
7
8      ldr r1, =i              @ r1 = direccion base del arreglo
9      mov r2, #0              @ r2 = contador (valor a guardar)
10     mov r4, #32             @ total de datos
11
12     loop:
13     cmp r2, r4              @ r2 == 32?
14     beq fin
15
16     str r2, [r1], #4        @ Guardar r2 y avanzar puntero 4 bytes
17     @ (direccionamiento indexado post-indexado)
18
19     add r2, r2, #1          @ r2++
20     b loop
21
22     fin:
23     mov r7, #1
24     svc 0

```

```

(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) n
Program not restarted.
(gdb) continue
Continuing.

Breakpoint 1, loop () at p2e2.s:19
19      add r2, r2, #1    @ r2++
(gdb) continue
Continuing.

Breakpoint 1, loop () at p2e2.s:19
19      add r2, r2, #1    @ r2++
(gdb) x /16xw 0x110a4
0x110a4:  0x00000001  0x00000002  0x00000000  0x00000000
0x110b4:  0x00000000  0x00000000  0x00000000  0x00000000
0x110c4:  0x00000000  0x00000000  0x00000000  0x00000000
0x110d4:  0x00000000  0x00000000  0x00000000  0x00000000
(gdb) continue
Continuing.

Breakpoint 1, loop () at p2e2.s:19
19      add r2, r2, #1    @ r2++
(gdb) x /16xw 0x110a4
0x110a4:  0x00000001  0x00000002  0x00000003  0x00000000
0x110b4:  0x00000000  0x00000000  0x00000000  0x00000000
0x110c4:  0x00000000  0x00000000  0x00000000  0x00000000
0x110d4:  0x00000000  0x00000000  0x00000000  0x00000000
(gdb) q

```

Figura 3: Debug Ejercicio 2 con dbg

r0	00000000	r0	00000000	r0	00000000	r0	00000000
r1	00000030	r1	00000030	r1	00000034	r1	00000034
r2	00000000	r2	00000000	r2	00000000	r2	00000001
r3	00000000	r3	00000000	r3	00000000	r3	00000000
r4	00000000	r4	00000020	r4	00000020	r4	00000020
r5	00000000	r5	00000000	r5	00000000	r5	00000000
r6	00000000	r6	00000000	r6	00000000	r6	00000000
r7	00000000	r7	00000000	r7	00000000	r7	00000000
r8	00000000	r8	00000000	r8	00000000	r8	00000000
r9	00000000	r9	00000000	r9	00000000	r9	00000000
r10	00000000	r10	00000000	r10	00000000	r10	00000000
r11	00000000	r11	00000000	r11	00000000	r11	00000000
r12	00000000	r12	00000000	r12	00000000	r12	00000000
sp	00000000	sp	00000000	sp	00000000	sp	00000000
lr	00000000	lr	00000000	lr	00000000	lr	00000000
pc	00000004	pc	0000000c	pc	00000018	pc	0000001c
cpsr	000001d3 NZCVI SVC	cpsr	000001d3 NZCVI SVC	cpsr	800001d3 NZCVI SVC	cpsr	800001d3 NZCVI SVC
spsr	00000000 NZCVI ?	spsr	00000000 NZCVI ?	spsr	00000000 NZCVI ?	spsr	00000000 NZCVI ?

Figura 4: CPULator Ejercicio 2

2.2.3. Analisis de Resultados

El programa inicializo un bloque de 128 bytes para almacenar 32 enteros del 0 al 31. Se utilizo direccionamiento post-indexado (str r2, [r1], #4), que permitio guardar el dato y actualizar automaticamente el puntero de memoria en una sola instruccion. Esto optimizo el ciclo al eliminar el calculo manual de desplazamientos en cada iteracion, utilizado en el ejercicio anterior, logrando una mejor gestion de memoria y la terminacion del programa mediante llamadas al sistema.

2.3. Ejercicio 3

Realizar un programa almacene en memoria un arreglo de datos de 32 bits con 16 elementos; una vez transferidos, realizar la copia en sentido inverso en otro arreglo.

A = [dato1, dato2, dato3, dato4,, dato15, dato16] @Original

B = [dato16, dato15, dato14, dato13,, dato2, dato1] @Copia

2.3.1. Pseudo codigo

Algoritmo 3: Copia de arreglo en orden inverso

```

Definir ORIGEN[16];
Definir DESTINO[16];
CONTADOR ← 0;
Mientras CONTADOR < 16 hacer;
    ORIGEN[CONTADOR] ← CONTADOR;
    CONTADOR ← CONTADOR + 1;
CONTADOR ← 0;
Mientras CONTADOR < 16 hacer;
    DESTINO[CONTADOR] ← ORIGEN[15 - CONTADOR];
    CONTADOR ← CONTADOR + 1;
Finalizar programa;

```

2.3.2. Desarrollo

```

1  .data
2  origen:  .skip 64          @ reservar 64 bytes (16 enteros de 32 bits)
3  destino: .skip 64          @ reservar 64 bytes para el arreglo destino
4
5  .text
6  .global _start
7  _start:
8
9  ldr r0, =origen            @ r0 <-- direccion base de origen
10 ldr r1, =destino           @ r1 <-- direccion base de destino
11 mov r2, #0                 @ r2 <-- contador
12 mov r3, #16                @ r3 <-- numero total de elementos
13
14 llenar:
15 cmp r2, r3                 @ comparar contador con 16
16 beq copiar                 @ si r2 == 16 pasar a copiar
17
18 str r2, [r0, r2, LSL #2]    @ origen[r2] = r2
19 add r2, r2, #1             @ r2 = r2 + 1
20 b llenar                   @ repetir ciclo
21
22 copiar:
23 mov r2, #0                 @ reiniciar contador
24
25 loop_inv:

```

```

26  cmp r2, r3           @ comparar contador con 16
27  beq fin             @ si r2 == 16 terminar
28
29  sub r4, r3, r2       @ r4 = 16 - r2
30  sub r4, r4, #1       @ r4 = indice inverso
31  ldr r5, [r0, r4, LSL #2] @ cargar origen[indice inverso]
32  str r5, [r1, r2, LSL #2] @ guardar en destino[r2]
33
34  add r2, r2, #1       @ r2 = r2 + 1
35  b loop_inv          @ repetir ciclo
36
37  fin:
38  mov r7, #1           @ syscall exit
39  svc 0

```

```

Starting program: /home/pi/Downloads/p2e3
Breakpoint 1, fin () at p2e3.s:38
38      mov r7, #1
(gdb) i r
r0      0x110cc      69836
r1      0x1110c      69900
r2      0x10        16
r3      0x10        16
r4      0x0         0
r5      0x0         0
r6      0x0         0
r7      0x0         0
r8      0x0         0
r9      0x0         0
r10     0x0         0
r11     0x0         0
r12     0x0         0
sp      0xffffef0c0  0xffffef0c0
lr      0x0         0
pc      0x100bc      0x100bc <fin>
cpsr    0x60000010  1610612752
fpscr   0x0         0
tpidruro <unavailable>
(gdb) x /16xw 0x1110c
0x1110c: 0x0000000f  0x0000000e  0x0000000d  0x0000000c
0x1111c: 0x0000000b  0x0000000a  0x00000009  0x00000008
0x1112c: 0x00000007  0x00000006  0x00000005  0x00000004
0x1113c: 0x00000003  0x00000002  0x00000001  0x00000000
(gdb)

```

Figura 5: Ejercicio 3 Debugado con dbg

r0	00000058				
r1	00000000				
r2	00000000				
r3	00000000				
r4	00000000				
r5	00000000				
r6	00000000				
r7	00000000				
r8	00000000				
r9	00000000				
r10	00000000				
r11	00000000				
r12	00000000				
sp	00000000				
lr	00000000				
pc	00000004				
cpsr	000001d3	NZCVI	SVC		
spsr	00000000	NZCVI	?		

r0	00000058				
r1	00000098				
r2	00000000				
r3	00000000				
r4	00000000				
r5	00000000				
r6	00000000				
r7	00000000				
r8	00000000				
r9	00000000				
r10	00000000				
r11	00000000				
r12	00000000				
sp	00000000				
lr	00000000				
pc	00000008				
cpsr	000001d3	NZCVI	SVC		
spsr	00000000	NZCVI	?		

r0	00000058				
r1	00000098				
r2	00000001				
r3	00000010				
r4	00000000				
r5	00000000				
r6	00000000				
r7	00000000				
r8	00000000				
r9	00000000				
r10	00000000				
r11	00000000				
r12	00000000				
sp	00000000				
lr	00000000				
pc	00000020				
cpsr	800001d3	NZCVI	SVC		
spsr	00000000	NZCVI	?		

r0	00000058				
r1	00000098				
r2	0000000a				
r3	00000010				
r4	00000000				
r5	00000000				
r6	00000000				
r7	00000000				
r8	00000000				
r9	00000000				
r10	00000000				
r11	00000000				
r12	00000000				
sp	00000000				
lr	00000000				
pc	00000020				
cpsr	800001d3	NZCVI	SVC		
spsr	00000000	NZCVI	?		

Figura 6: CPULATOR Ejercicio 3

2.3.3. Analisis de Resultados

El programa realizo la transferencia de 16 enteros entre dos bloques de memoria, implementando una inversion de orden. Mediante el calculo de un indice inverso ($15 - n$) y direccionamiento indexado con desplazamiento, el algoritmo extrajo los datos del bloque de origen de forma descendente y los almaceno en el destino de forma ascendente. El resultado final confirmo la correcta manipulacion de punteros y la preservacion de la alineacion de 32 bits en operaciones de transferencia.

2.4. Ejercicio 4

Realizar un programa que forme un arreglo de 20 elementos, con el siguiente criterio:

$$A = [i, 2i, 4i, 8i, 16i, \dots, ni]$$

2.4.1. Pseudo codigo

Algoritmo 4: Generacion de arreglo con numeros pares

```
Definir ARREGLO[20];
VALOR  $\leftarrow$  0;
INDICE  $\leftarrow$  0;
Mientras INDICE < 20 hacer;
    ARREGLO[INDICE]  $\leftarrow$  VALOR;
    VALOR  $\leftarrow$  VALOR + 2;
    INDICE  $\leftarrow$  INDICE + 1;
Finalizar programa;
```

2.4.2. Desarrollo

```
1  .data
2  arreglo: .skip 80          @ 20 elementos de 32 bits
3
4  .text
5  .global _start
6  _start:
7
8  ldr r0, =arreglo
9  mov r1, #0                @ contador
10 mov r2, #0                @ valor
11 mov r3, #20
12
13 loop:
14 cmp r1, r3
15 beq fin
16
17 str r2, [r0, r1, LSL #2]
18 add r2, r2, #2            @ genera numeros pares
19 add r1, r1, #1
20 b loop
21
22 fin:
23 mov r7, #1
24 svc 0
```

```
(gdb) r
Starting program: /home/pi/Downloads/p2e4

Breakpoint 1, loop () at p2e4.s:20
20      b loop
(gdb) i r
r0      0x110a8      69800
r1      0x1         1
r2      0x2         2
r3      0x14        20
r4      0x0         0
r5      0x0         0
r6      0x0         0
r7      0x0         0
r8      0x0         0
r9      0x0         0
r10     0x0         0
r11     0x0         0
r12     0x0         0
sp      0xffffef0c0 0xffffef0c0
lr      0x0         0
pc      0x10098      0x10098 <loop+20>
cpsr    0x80000010  -2147483632
fpscr   0x0         0
tpidru0 <unavailable>
(gdb)
```

Figura 7: Ejercicio 4 Debugado con dbg

r0	00000038		r0	00000038		r0	00000038		r0	00000038	
r1	00000000		r1	00000000		r1	00000000		r1	00000006	
r2	00000000		r2	00000000		r2	00000002		r2	0000000e	
r3	00000000		r3	00000014		r3	00000014		r3	00000014	
r4	00000000		r4	00000000		r4	00000000		r4	00000000	
r5	00000000		r5	00000000		r5	00000000		r5	00000000	
r6	00000000		r6	00000000		r6	00000000		r6	00000000	
r7	00000000		r7	00000000		r7	00000000		r7	00000000	
r8	00000000		r8	00000000		r8	00000000		r8	00000000	
r9	00000000		r9	00000000		r9	00000000		r9	00000000	
r10	00000000		r10	00000000		r10	00000000		r10	00000000	
r11	00000000		r11	00000000		r11	00000000		r11	00000000	
r12	00000000		r12	00000000		r12	00000000		r12	00000000	
sp	00000000		sp	00000000		sp	00000000		sp	00000000	
lr	00000000		lr	00000000		lr	00000000		lr	00000000	
pc	00000004		pc	00000010		pc	00000020		pc	00000020	
cpsr	000001d3	NZCVI SVC	cpsr	000001d3	NZCVI SVC	cpsr	800001d3	NZCVI SVC	cpsr	800001d3	NZCVI SVC
spsr	00000000	NZCVI ?	spsr	00000000	NZCVI ?	spsr	00000000	NZCVI ?	spsr	00000000	NZCVI ?

Figura 8: CPULator Ejercicio 4

00000030	00000038	00000000	00000000	00000002
00000040	00000004	00000006	00000008	0000000a
00000050	0000000c	0000000e	00000010	00000012
00000060	00000000	00000000	00000000	00000000
00000070	00000000	00000000	00000000	00000000
00000080	00000000	00000000	aaaaaaaa	aaaaaaaa

Figura 9: Memoria CPULator Ejercicio 4

2.4.3. Analisis de Resultados

El programa inicializo un bloque de 80 bytes con una sucesion aritmetica de 20 numeros pares del 0 al 38. Ademas, se utilizo una logica con dos punteros: uno para controlar la iteracion y otro para generar el numero par, empleando direccionamiento indexado con desplazamiento para garantizar la alineacion de cada dato de 32 bits en memoria. El proceso finalizo exitosamente tras completar el llenado del arreglo mediante una salida controlada por el sistema.

2.5. Ejercicio 5

Realizar un programa que multiplique dos matrices de 2x2; los datos podran ser de 8 bits.

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} I & J \\ K & L \end{bmatrix}$$

2.5.1. Pseudo codigo

Algoritmo 5: Multiplicacion de matrices 2x2

```

Definir matrices A[2][2], B[2][2];
Definir matriz C[2][2];
C[0][0] ← A[0][0] * B[0][0] + A[0][1] * B[1][0];
C[0][1] ← A[0][0] * B[0][1] + A[0][1] * B[1][1];
C[1][0] ← A[1][0] * B[0][0] + A[1][1] * B[1][0];
C[1][1] ← A[1][0] * B[0][1] + A[1][1] * B[1][1];
Finalizar programa;
  
```

2.5.2. Desarrollo

```

1  .data
2  A: .byte 1,2,3,4
3  B: .byte 5,6,7,8
4  C: .space 4
5
6  .text
7  .global _start
8  _start:
9
10 ldr r0, =A
11 ldr r1, =B
12 ldr r2, =C
13
14 ldrb r3, [r0]           @ A
15 ldrb r4, [r0, #1]      @ B
16 ldrb r5, [r0, #2]      @ C
17 ldrb r6, [r0, #3]      @ D
18
19 ldrb r7, [r1]           @ E
20 ldrb r8, [r1, #1]      @ F
21 ldrb r9, [r1, #2]      @ G
22 ldrb r10, [r1, #3]     @ H
23
24 mul r11, r3, r7
25 mul r12, r4, r9
26 add r11, r11, r12
27 strb r11, [r2]
28
29 mul r11, r3, r8
30 mul r12, r4, r10
31 add r11, r11, r12
32 strb r11, [r2, #1]
33
34 mul r11, r5, r7
35 mul r12, r6, r9
  
```

```
36     add r11, r11, r12
37     strb r11, [r2, #2]
38
39     mul r11, r5, r8
40     mul r12, r6, r10
41     add r11, r11, r12
42     strb r11, [r2, #3]
43
44     fin:
45     mov r7, #1
46     svc 0
```

```
(gdb) run
Starting program: /home/pi/Downloads/p2e5

Breakpoint 1, fin () at p2e5.s:45
45     mov r7, #1
(gdb) i r
r0          0x110f4          69876
r1          0x110f8          69880
r2          0x110fc          69884
r3          0x1             1
r4          0x2             2
r5          0x3             3
r6          0x4             4
r7          0x5             5
r8          0x6             6
r9          0x7             7
r10         0x8             8
r11         0x32            50
r12         0x20            32
sp          0xffffef0c0     0xffffef0c0
lr          0x0             0
pc          0x100e0         0x100e0 <fin>
cpsr       0x10            16
fpscr      0x0             0
tpidruro   <unavailable>
(gdb) █
```

Figura 10: Ejercicio 5 Debugado con dbg

r0	00000080	
r1	00000084	
r2	00000000	
r3	00000000	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000088	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

r0	00000080	
r1	00000084	
r2	00000088	
r3	00000001	
r4	00000002	
r5	00000003	
r6	00000004	
r7	00000005	
r8	00000006	
r9	00000007	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000028	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

r0	00000080	
r1	00000084	
r2	00000088	
r3	00000001	
r4	00000002	
r5	00000003	
r6	00000004	
r7	00000005	
r8	00000006	
r9	00000007	
r10	00000008	
r11	00000013	
r12	0000000e	
sp	00000000	
lr	00000000	
pc	00000038	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

r0	00000080	
r1	00000084	
r2	00000088	
r3	00000001	
r4	00000002	
r5	00000003	
r6	00000004	
r7	00000005	
r8	00000006	
r9	00000007	
r10	00000008	
r11	00000012	
r12	00000020	
sp	00000000	
lr	00000000	
pc	00000064	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

Figura 11: CPULator Ejercicio 5

2.5.3. Analisis de Resultados

El programa implemento exitosamente una multiplicacion de matrices de 2×2 utilizando aritmetica de 8 bits. Mediante el uso de instrucciones de carga y almacenamiento de bytes, con `ldrb` y `strb`, se ejecuto el producto punto entre las filas de la matriz A y las columnas de la matriz B. El resultado valido la correcta gestion de desplazamientos inmediatos en memoria y la capacidad del procesador para realizar operaciones combinadas de multiplicacion (`mul`) y suma (`add`) sobre registros independientes para obtener una matriz resultante C de 4 elementos.

2.6. Ejercicio 6

Realizar un programa que encuentre el numero con valor mayor en un arreglo de 20 elementos que seran almacenados en memoria; para lo cual:

- Indicar cual fue el valor mayor.
- Ubicar la direccion donde se encontro este numero.
- Usar las direcciones que se requieran para cumplir lo solicitado.

2.6.1. Pseudo codigo

Algoritmo 6: Busqueda del valor mayor en un arreglo

```

Definir ARREGLO[20];
MAYOR  $\leftarrow$  ARREGLO[0];
DIRECCION  $\leftarrow$  0;
INDICE  $\leftarrow$  1;
Mientras INDICE < 20 hacer;
    Si ARREGLO[INDICE] > MAYOR entonces;
        MAYOR  $\leftarrow$  ARREGLO[INDICE];
        DIRECCION  $\leftarrow$  INDICE;
    INDICE  $\leftarrow$  INDICE + 1;
Mostrar MAYOR;
Mostrar DIRECCION;
Finalizar programa;

```

2.6.2. Desarrollo

```

1  .data
2  arreglo: .word 5,8,3,10,2,7,15,1,4,6,9,11,14,13,12,0,16,18,17,19
3  mayor:   .word 0
4  dir:     .word 0
5
6  .text
7  .global _start
8  _start:
9
10 ldr r0, =arreglo
11 ldr r1, [r0]           @ primer valor
12 mov r2, #1
13 mov r3, #20
14 mov r4, r0             @ direccion mayor
15
16 loop:
17 cmp r2, r3
18 beq guardar
19
20 ldr r5, [r0, r2, LSL #2]
21 cmp r5, r1
22 ble siguiente
23
24 mov r1, r5
25 add r4, r0, r2, LSL #2
26

```

```

27     siguiente:
28     add r2, r2, #1
29     b loop
30
31     guardar:
32     ldr r6, =mayor
33     str r1, [r6]
34
35     ldr r6, =dir
36     str r4, [r6]
37
38     fin:
39     mov r7, #1
40     svc 0

```

```

(gdb) r
Starting program: /home/pi/Downloads/p2e6

Breakpoint 1, fin () at p2e6.s:39
39     mov r7, #1
(gdb) i r
r0          0x110d0          69840
r1          0x13            19
r2          0x14            20
r3          0x14            20
r4          0x1111c         69916
r5          0x13            19
r6          0x11124         69924
r7          0x0             0
r8          0x0             0
r9          0x0             0
r10         0x0             0
r11         0x0             0
r12         0x0             0
sp          0xffffef0c0     0xffffef0c0
lr          0x0             0
pc          0x100bc         0x100bc <fin>
cpsr       0x60000010      1610612752
fpscr      0x0             0
tpidruro   <unavailable>
(gdb) x /16xw 0x11124
0x11124:  0x0001111c  0x00001141  0x61656100  0x01006962
0x11134:  0x00000007  0x00000108  0x00000000  0x0000001c
0x11144:  0x00000002  0x00040000  0x00000000  0x00010074
0x11154:  0x0000005c  0x00000000  0x00000000  0x00000022
(gdb)

```

Figura 12: Ejercicio 6 Debugeado con dbg

r0	00000060	
r1	00000005	
r2	00000001	
r3	00000014	
r4	00000000	
r5	00000000	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000010	
cpsr	000001d3	NZCVI SVC
spsr	00000000	NZCVI ?

r0	00000060	
r1	00000005	
r2	00000001	
r3	00000014	
r4	00000060	
r5	00000008	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000020	
cpsr	800001d3	NZCVI SVC
spsr	00000000	NZCVI ?

r0	00000060	
r1	00000008	
r2	00000001	
r3	00000014	
r4	00000060	
r5	00000008	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	0000002c	
cpsr	200001d3	NZCVI SVC
spsr	00000000	NZCVI ?

r0	00000060	
r1	00000008	
r2	00000003	
r3	00000014	
r4	00000064	
r5	00000003	
r6	00000000	
r7	00000000	
r8	00000000	
r9	00000000	
r10	00000000	
r11	00000000	
r12	00000000	
sp	00000000	
lr	00000000	
pc	00000034	
cpsr	800001d3	NZCVI SVC
spsr	00000000	NZCVI ?

Figura 13: CPULator Ejercicio 6

2.6.3. Analisis de Resultados

El programa implemento un algoritmo de busqueda del valor mayor en un arreglo de 20 elementos de 32 bits. Mediante comparaciones iterativas y saltos condicionales (ble), el codigo fue eligiendo entre los valores del arreglo, almacenando el numero mas alto y su direccion de memoria correspondiente en dos registros diferentes. Finalmente, el resultado valido la correcta implementacion de estructuras de control y el manejo de punteros dinamicos para la extraccion de informacion en tiempo de ejecucion.


```

14  @ ----- COPIAR -----
15  copiar:
16  cmp r2, r3
17  beq ordenar
18
19  ldr r4, [r0, r2, LSL #2]
20  str r4, [r1, r2, LSL #2]
21  add r2, r2, #1
22  b copiar
23
24  @ ----- ORDENAR -----
25  ordenar:
26  mov r2, #0          @ i = 0
27
28  outer:
29  cmp r2, r3
30  beq fin
31
32  mov r4, #0          @ j = 0
33  sub r8, r3, #1      @ limite = n - 1
34
35  inner:
36  cmp r4, r8          @ j < n-1
37  bge next
38
39  ldr r5, [r1, r4, LSL #2] @ A[j]
40
41  add r9, r4, #1
42  ldr r6, [r1, r9, LSL #2] @ A[j+1]
43
44  cmp r5, r6
45  ble skip
46
47  str r6, [r1, r4, LSL #2]
48  str r5, [r1, r9, LSL #2]
49
50  skip:
51  add r4, r4, #1
52  b inner
53
54  next:
55  add r2, r2, #1
56  b outer
57
58  @ ----- FIN -----
59  fin:
60  mov r7, #1
61  svc 0

```

```

Breakpoint 2, fin () at p2e7PRUEBA.s:60
60      mov r7, #1
(gdb) i r
r0      0x110f4      69876
r1      0x11174      70004
r2      0x20        32
r3      0x20        32
r4      0x1f        31
r5      0x1e        30
r6      0x1f        31
r7      0x0         0
r8      0x1f        31
r9      0x1f        31
r10     0x0         0
r11     0x0         0
r12     0x0         0
sp      0xffffef0b0  0xffffef0b0
lr      0x0         0
pc      0x100e4      0x100e4 <fin>
cpsr    0x60000010  1610612752
fpscr   0x0         0
tpidruo <unavailable>
(gdb) x /16xw 0x11174
0x11174:  Cannot access memory at address 0x11174
(gdb) x /16xw 0x11174
0x11174:  0x00000000  0x00000001  0x00000002  0x00000003
0x11184:  0x00000004  0x00000005  0x00000006  0x00000007
0x11194:  0x00000008  0x00000009  0x0000000a  0x0000000b
0x111a4:  0x0000000c  0x0000000d  0x0000000e  0x0000000f
(gdb)

```

Figura 14: Ejercicio 7 Debugado con dbg

r0	00000080	r0	00000080	r0	00000080	r0	00000080
r1	00000100	r1	00000100	r1	00000100	r1	00000100
r2	00000000	r2	00000000	r2	00000000	r2	00000002
r3	00000000	r3	00000020	r3	00000020	r3	00000020
r4	00000000	r4	00000009	r4	00000000	r4	00000000
r5	00000000	r5	00000000	r5	00000009	r5	0000001e
r6	00000000	r6	00000000	r6	00000000	r6	0000001f
r7	00000000	r7	00000000	r7	00000000	r7	00000000
r8	00000000	r8	00000000	r8	0000001f	r8	0000001f
r9	00000000	r9	00000000	r9	00000000	r9	0000001f
r10	00000000	r10	00000000	r10	00000000	r10	00000000
r11	00000000	r11	00000000	r11	00000000	r11	00000000
r12	00000000	r12	00000000	r12	00000000	r12	00000000
sp	00000000	sp	00000000	sp	00000000	sp	00000000
lr	00000000	lr	00000000	lr	00000000	lr	00000000
pc	00000008	pc	0000001c	pc	00000048	pc	00000038
cpsr	000001d3 NZCVI SVC	cpsr	800001d3 NZCVI SVC	cpsr	800001d3 NZCVI SVC	cpsr	800001d3 NZCVI SVC
spsr	00000000 NZCVI ?	spsr	00000000 NZCVI ?	spsr	00000000 NZCVI ?	spsr	00000000 NZCVI ?

Figura 15: CPULATOR Ejercicio 7

2.7.3. Analisis de Resultados

El programa implemento el algoritmo Bubble Sort para ordenar 32 enteros de 32 bits. La ejecucion valido el uso de ciclos anidados y saltos condicionales para realizar comparaciones adyacentes y el intercambio de valores en memoria conocido como swapping. Al finalizar, se verifico que el bloque de memoria ordenado contenia la secuencia numerica en orden ascendente, demostrando una gestion correcta de los punteros y un control adecuado del flujo del programa solicitado.

3. Conclusiones

El desarrollo de esta práctica se llevó a cabo gracias al tutorial de raspberry pi de la facultad [1] y a las clases del profesor Moisés [2].

3.1. Hernandez Acosta Mauricio Gabriel

Durante el desarrollo de esta práctica, logré consolidar mis conocimientos sobre el manejo directo de la memoria en la arquitectura ARM. La resolución de los ejercicios me permitió comprobar la importancia de estructurar correctamente los ciclos y las ramificaciones condicionales para procesar arreglos y matrices sin errores lógicos. El uso del direccionamiento indexado con desplazamiento (offset) resultó ser una herramienta clave para recorrer y manipular bloques de datos de 32 bits de manera precisa. En conclusión, esta experiencia fue fundamental para entender que la programación a bajo nivel requiere una orquestación cuidadosa de los registros para que actúen como punteros efectivos.

3.2. Perez Olvera Alexis Abraham

La realización de estas actividades evidenció la enorme ventaja que ofrecen los modos de direccionamiento indirecto para la optimización del código en lenguaje ensamblador. Al programar los algoritmos de búsqueda y ordenamiento, comprobé que técnicas como el direccionamiento post-indexado y los desplazamientos lógicos (LSL) reducen drásticamente la cantidad de instrucciones necesarias para iterar sobre grandes estructuras de datos. Entender cómo el procesador gestiona y actualiza las direcciones de memoria de forma automática me brindó la perspectiva necesaria para escribir rutinas que no solo sean funcionales, sino también compactas y eficientes en su ejecución.

3.3. Sierra Garcia Mariana

Esta práctica representó un excelente ejercicio para trasladar la teoría de estructuras de datos a su aplicación real a nivel de hardware. Pude asimilar cómo operaciones que parecen complejas, tales como la multiplicación de matrices, la generación de secuencias o el ordenamiento de elementos, se resuelven netamente mediante la aritmética de punteros. Emplear las distintas variantes de direccionamiento y desplazamientos con LSL me ayudó a dominar el acceso secuencial a la memoria. En resumen, comprendí que la correcta administración y actualización de los registros es la base para interactuar exitosamente con cualquier bloque de información en el sistema.

Referencias

- [1] R. Anaya García et al., *Manual de Aplicaciones Raspberry Pi*, Manual de recursos y aplicaciones para plataforma Raspberry Pi, Universidad Nacional Autónoma de México, Facultad de Ingeniería, Laboratorio de Microcomputadoras, Ciudad Universitaria Coyoacán, México, 2026. dirección: <https://odin.fi-b.unam.mx/micros/docs/Tutoriales%20Raspberry.pdf>
- [2] M. Meléndez Reyes, *Notas de clase: Práctica 01 - Introducción a las arquitecturas ARM empleando Raspberry Pi*, Clase presencial en laboratorio de microprocesadores, Facultad de Ingeniería, UNAM, Material proporcionado por el profesor durante la sesión de laboratorio, feb. de 2026.